ICPC-JRC INTRA UAP PROGRAMMING CONTEST 2026

Junior Division — Official Problem Set

Problem A: Back and Forth

A line of $n \geq 2$ cells is numbered 1 through n. A signal sits on cell 1 at time 0 and moves one cell to the right per step. When it would step past a border, it turns around instead, so it bounces between cell 1 and cell n forever.

For $n = 4$, the signal visits cells $1, 2, 3, 4, 3, 2, 1, 2, 3, 4, \ldots$ at times $0, 1, 2, 3, 4, 5, 6, 7, 8, 9, \ldots$

Which cell holds the signal at time t?

INPUT

The first line contains a single integer q ($1 \leq q \leq 10^5$) — the number of queries.
Each of the next q lines contains two integers n and t ($2 \leq n \leq 10^{18}$, $0 \leq t \leq 10^{18}$).

OUTPUT

For each query, print one integer — the cell holding the signal at time t.

EXAMPLE

Input	Output
5 8 3 8 11 2 7 3 4 1000000000000000000 1000000000000000000	4 4 2 1 999999999999999999

Notes:

- In the first two queries, the signal is on cells 1 through 8 at times 0 through 7, then turns around and is on cells $7, 6, 5, 4$ at times $8, 9, 10, 11$.
- In the third query, the signal has only two cells to work with, so it alternates $1, 2, 1, 2, \ldots$ and lands on cell 2 at every odd time.
- In the fourth query, the signal reaches cell 3 at time 2, turns around, and is back on cell 1 at time 4.

Problem B: The Sixfold Signal

In a futuristic communication system, every signal is assigned a numerical strength level. Engineers have discovered that a valid signal must follow a special pattern: starting from the initial strength of 1, the signal strength must be multiplied by 6 at every stage.

Thus, the valid signal strengths are: $1, 6, 36, 216, 1296, \ldots$

Given an integer n, determine whether it represents a valid signal strength. In other words, n is a valid signal strength if there exists a non-negative integer x such that $n = 6^x$.

INPUT

The input contains a single integer n.

CONSTRAINTS

$-2^{31} \le n \le 2^{31} - 1$

OUTPUT

Print YES if n is a power of 6; otherwise, print NO.

EXAMPLES

Sample Input 1	Sample Output 1
36	YES

Sample Input 2	Sample Output 2
12	NO

Problem C: DySec - Malicious Package Hunt

Thousands of new Python packages are published to PyPI every day. Some appear entirely ordinary on inspection, yet begin to exhibit malicious behaviour the moment they are installed. DySec is a dynamic analysis system that observes a package while it executes and records the behavioural traces it produces.

DySec monitors six categories of behavioural events:

Behavioural Event	Code
File activity	F
Open / file access	O
Installation activity	I
Network / TCP activity	T
System call activity	S
Suspicious execution pattern	P

Every observed event carries an associated risk score.

PROBLEM DESCRIPTION

Executing a package produces a sequence of N events, given in chronological order. A single suspicious event is not sufficient evidence of malice — many perfectly benign packages open files, issue system calls, or contact the network during installation. DySec therefore looks for concentrated suspicious behaviour.

Specifically, a contiguous window of events is called a **Malicious Behavioural Window** if it satisfies both of the following conditions:

1. It contains at least K distinct behavioural categories, and
2. The total risk score of its events is at least R.

Given the event trace, determine the shortest Malicious Behavioural Window.

INPUT

The first line contains three integers: N, K, and R.
Each of the next N lines describes one event in the format: `type score`

- `type` — a single character from `{F, O, I, T, S, P}`
- `score` — the risk score of that event

CONSTRAINTS

- $1 \le N \le 2 \times 10^5$
- $1 \le K \le 6$
- $1 \le R \le 10^9$
- $1 \le score \le 10^6$
- All event indices are 1-based.
- All risk scores are strictly positive; consequently, the total score of a window never decreases as the window is extended.

OUTPUT

If at least one Malicious Behavioural Window exists, print three integers separated by spaces:
`length start end`

- `length` — the size of the shortest such window
- `start` — the 1-based index of its first event
- end — the 1-based index of its last event

If several windows share the minimum length, print the one with the smallest starting index.

If no Malicious Behavioural Window exists, print: SAFE

EXAMPLE

Input	Output
10 4 20 F 3 S 5 F 2 T 6 O 4 S 1 F 2 I 3 I 4	5 1 5

Problem D: The Professor's Browser History

Late at night before a big lecture, a professor is furiously browsing through research pages, jumping back and forth between tabs like everyone does at 2 AM. You've been asked to build a tiny browser simulator that replays his session, since he keeps losing track of which page he's actually on right now.

The browser starts on page 0 (the homepage), with both the back-history and forward-history empty. You will process q operations, each of which is one of the following four types:

1. `VISIT x` — Navigate to page x. The current page is pushed onto the back history, page x becomes the new current page, and the entire forward history is cleared. This is standard browser behavior.
2. `BACK` — Move back one page, if possible. The current page is pushed onto the forward history, and the most recent page from the back history becomes the current page. If the back history is empty, nothing happens.
3. `FORWARD` — Move forward one page, if possible. The current page is pushed onto the back history, and the most recent page from the forward history becomes the current page. If the forward history is empty, nothing happens.
4. `CURRENT?` — Print the page number the professor is currently viewing.

INPUT

- The first line contains an integer q, the number of operations.
- The next q lines each contain one operation in one of the formats described above.
- For `VISIT x`, x satisfies $0 \le x \le 10^9$ and x is guaranteed to be different from the current page.

CONSTRAINTS

- $1 \le q \le 2 \times 10^5$
- $0 \le x \le 10^9$

OUTPUT

For every `CURRENT?` operation, print the page number of the page the professor is currently viewing.

EXAMPLE

Input	Output
10 VISIT 5 VISIT 8 BACK VISIT 3 CURRENT? BACK BACK FORWARD CURRENT? FORWARD	3 5

Step-by-step Trace:

1. `VISIT 5` → current = ***5***, back = ***[0]***, forward = ***[]***
2. `VISIT 8` → current = ***8***, back = ***[0, 5]***, forward = ***[]***
3. `BACK` → current = ***5***, back = ***[0]***, forward = ***[8]***
4. `VISIT 3` → current = ***3***, back = ***[0, 5]***, forward = ***[]*** *(Forward history cleared)*
5. `CURRENT?` → Prints 3
6. `BACK` → current = ***5***, back = ***[0]***, forward = ***[3]***
7. `BACK` → current = ***0***, back = ***[]***, forward = ***[3, 5]***
8. `FORWARD` → current = ***5***, back = ***[0]***, forward = ***[3]***
9. `CURRENT?` → Prints 5
10. `FORWARD` → current = ***3***, back = ***[0, 5]***, forward = ***[]***

Problem E: The Professor's Signal Towers

Professor Bill Poucher is designing a communication network for a remote research facility. Along a long straight road, there are n possible locations where signal towers can be built.

The locations are given by their coordinates on the road. Due to interference between nearby towers, the network becomes more reliable when the selected towers are placed as far apart as possible.

The professor wants to build exactly k towers from the available locations. The quality of the network is determined by the minimum distance between any two selected towers.

Your task is to choose k tower locations so that this minimum distance is as large as possible. Print the maximum possible value of this minimum distance.

INPUT

- The first line contains two integers n and k — the number of available tower locations and the number of towers to build.
- The second line contains n integers $x_1, x_2, \ldots, x_n$ — the coordinates of the available locations.

CONSTRAINTS

- $2 \le k \le n \le 2 \times 10^5$
- $0 \le x_1 \le 10^9$
- $x_1 < x_2 < \ldots < x_n$

OUTPUT

Print a single integer representing the maximum possible value of the minimum distance between any two selected towers.

EXAMPLE

Input	Output
5 3 1 2 4 8 9	3

Notes:
The professor needs to build exactly 3 towers. One optimal choice is to build them at positions 1, 4, and 8.
The distances between consecutive selected towers are: $4 - 1 = 3$ and $8 - 4 = 4$.
Therefore, the minimum distance is 3.
It is not possible to choose 3 locations such that every pair of consecutive selected towers is at least 4 units apart.

Problem F: Closest Station

There are n bus stations located along a straight road. The coordinates of the stations are given in strictly increasing order.

You are given q independent queries. For each query, you are standing at position p on the road and want to find the nearest bus station.

If two stations are at the same distance from your position, choose the station with the smaller coordinate.

INPUT

- The first line contains two integers n and q, representing the number of bus stations and the number of queries.
- The second line contains n integers $x_1, x_2, \ldots, x_n$, representing the coordinates of the bus stations in strictly increasing order.
- The next q lines each contain one integer p, representing your position for that query.

CONSTRAINTS

- $1 \le n, q \le 2 \times 10^5$
- $-10^9 \le x_i \le 10^9$
- $x_1 < x_2 < \ldots < x_n$
- $-10^9 \le p \le 10^9$

OUTPUT

For each query, print the coordinate of the nearest bus station on a separate line. If two stations are equally close, print the smaller coordinate.

EXAMPLE

Input	Output
5 5 3 5 10 15 20 1 7 12 17 20	3 5 10 15 20

Problem G: A Tribute to the Architect of Bangladesh's IT Foundations

This problem has no input.

OUTPUT

For this problem, just print: `Long Live the Legacy of JRC Sir!` (without quotes).

Problem H: Square Balance

You are given a positive integer n.

A positive integer x is called **balanced** with n if:

$$lcm(n, x) = [gcd(n, x)]^2$$

Among all positive integers x that are balanced with n, find:

- The minimum possible value of x (x_{min}), and
- The total number of positive integers x that are balanced with n (c).

It can be proven that at least one balanced integer exists for every positive integer n.

INPUT

The first line contains an integer t — the number of test cases.
Each of the next t lines contains one positive integer n.

CONSTRAINTS

- $1 \le t \le 10^4$
- $1 \le n \le 10^9$
- It is guaranteed that the minimum valid value of x fits in a signed 64-bit integer.

OUTPUT

For each test case, print two integers separated by a space: `xmin c`

EXAMPLE

Input	Output
6 1 2 4 12 72 360	1 1 4 1 2 2 18 2 192 2 4800 2

Notes & Explanation:

For $n = 4$, the valid values of x are 2 and 16.

- For $x = 2$: $gcd(4, 2) = 2$ and $lcm(4, 2) = 4 = 2^2$
- For $x = 16$: $gcd(4, 16) = 4$ and $lcm(4, 16) = 16 = 4^2$

Therefore, the minimum valid value is 2, and there are exactly 2 valid values.